

CCS2313 - PROJECT MANAGEMENT

JustAsk LK

An anonymous, AI-first health support platform for Sri Lanka . Final project documentation covering planning, execution, risk management, and project closure.

Project Team

Team Member	Student ID	Role
Kavindu Mihiranga	CIT-24-02-0094	AI / Conversation Engineer
H. C. Shakya	CIT-24-02-0149	Safety Engineer
Dinuvi Laksarani	CIT-24-02-0049	Frontend / UI Developer
Himasha Dushan	CIT-24-02-0050	Backend / Data Engineer
Chanuka Lakshan	CIT-24-02-0069	UX and Documentation Lead

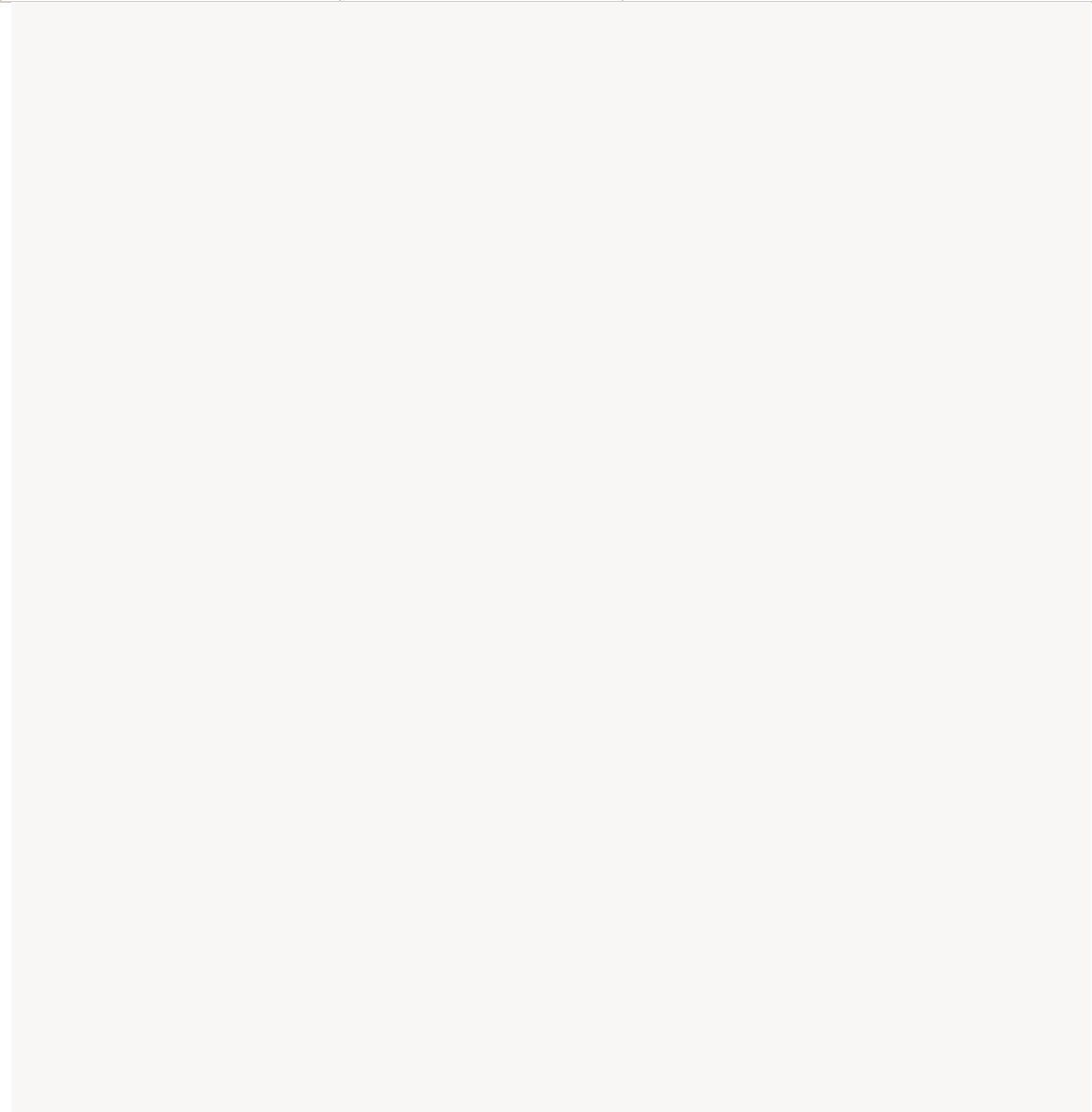


Table of Contents

Project Team	1
Table of Contents	2
1. Project Plan	3
1.1 Work Breakdown Structure (WBS)	3
1.2 Effort Estimation	5
1.3 Risk Assessment and Mitigation Strategies	6
1.4 Cost Estimation	7
1.4 Resource Allocation.....	8
1.5 Communication and Reporting Plan.....	9
2. Project Execution and Monitoring	10
2.1 Issue Tracking and Change Management	10
2.2 Status Updates and Team Meetings	12
3. Project Closure Report	13
3.1 Summary of Project Execution	13
3.2 Lessons Learned and Challenges Faced	14
3.3 Team Reflections and Individual Contributions	15
3.4 Future Recommendations	18
4. Appendices.....	19
Appendix A – Links.....	19
Appendix B - Supporting Documents	20
Appendix C - Verifying the Anonymity and Encryption Claims.....	20

1. Project Plan

1.1 Work Breakdown Structure (WBS)

The project is decomposed to level 4. Level 1 is the project, level 2 the phases, level 3 the deliverables, and level 4 the individual tasks. Every level-4 task is assigned to a named owner. This decomposition provides full traceability from the highest-level project objective down to the smallest unit of assignable work, enabling precise effort tracking and accountability throughout the project lifecycle.

WBS	Task	Owner	Deliverable
1	JustAsk LK Platform	All	--
1.1	Planning and Setup		
1.1.1	Requirements and scope definition	All	Project proposal
1.1.2	Legal and ethical review (PDPA, SLMC, Medical Ordinance)	Chanuka Lakshan	Proposal sections 5-6
1.1.3	Technology stack selection	Himasha Dushan	Proposal section 10
1.1.4	Repository and branch strategy setup	Himasha Dushan	GitHub repository
1.1.5	System architecture design	Himasha Dushan	Architecture diagram
1.2	Backend Foundations		
1.2.1	Express server and MongoDB connection	Himasha Dushan	index.js
1.2.2	Anonymous session schema (no identity fields)	Himasha Dushan	models/session.js
1.2.3	Session creation and update endpoints	Himasha Dushan	routes/session.js
1.2.4	Chat message schema with 30-day expiry	Himasha Dushan	models/chatMessage.js
1.3	AI Conversation Engine		
1.3.1	LLM provider evaluation and selection	Kavindu Mihiranga	Gemini API
1.3.2	Safety system prompt authoring	Kavindu Mihiranga	services/aiEngine.js
1.3.3	Conversation history and context handling	Kavindu Mihiranga	aiEngine.js
1.3.4	Model fallback chain and graceful degradation	Kavindu Mihiranga	aiEngine.js
1.3.5	Tone and conversation-stage tuning	Kavindu Mihiranga	aiEngine.js

1.4	Crisis Safety Layer		
1.4.1	English crisis pattern definition	H. C. Shakya	services/safetyLayer.js
1.4.2	Sinhala crisis pattern definition	H. C. Shakya	safetyLayer.js
1.4.3	Helpline routing incl. under-16 to Childline	H. C. Shakya	safetyLayer.js
1.4.4	Test set construction and execution	H. C. Shakya	safetyLayer.test.js
1.5	Doctor System		
1.5.1	Doctor registration with password hashing	Himasha Dushan	routes/doctorAuth.js
1.5.2	Authentication and verification middleware	Himasha Dushan	middleware/auth.js
1.5.3	Session queue and atomic claim	Himasha Dushan	routes/doctor.js
1.5.4	Doctor dashboard interface	Dinuvi Laksarani	DoctorDashboard.jsx
1.5.5	Administrator verification interface	Dinuvi Laksarani	AdminDashboard.jsx
1.6	Frontend		
1.6.1	Landing page (bilingual)	Dinuvi Laksarani	public/landing.html
1.6.2	Age band and topic selection screens	Dinuvi Laksarani	AgeSelect / TopicSelect
1.6.3	Chat interface with crisis banner	Dinuvi Laksarani	ChatScreen.jsx
1.6.4	Sinhala localisation and font handling	Chanuka Lakshan	LangContext.jsx
1.6.5	Voice input (browser speech-to-text)	Dinuvi Laksarani	useSpeechInput.js
1.6.6	Dark mode and responsive layout	Dinuvi Laksarani	index.css
1.7	Security Hardening		
1.7.1	Encryption at rest (AES-256-GCM)	Himasha Dushan	services/encryption.js
1.7.2	Audit logging of privileged actions	Himasha Dushan	services/audit.js
1.7.3	Rate limiting and account lockout	H. C. Shakya	rateLimit.js, loginGuard.js
1.7.4	Session ownership tokens	Himasha Dushan	middleware/sessionAuth.js
1.7.5	Session timeout sweeper	Himasha Dushan	sessionTimeout.js
1.8	Testing and Closure		
1.8.1	Crisis detection test execution	H. C. Shakya	Test results
1.8.2	Cross-device and browser testing	Dinuvi Laksarani	Test log
1.8.3	Deployment to production hosting	Himasha Dushan	Live URLs

1.8.4	Final documentation	Chanuka Lakshan	This document
1.8.5	Demonstration rehearsal	All	Demo script

1.2 Effort Estimation

Technique used: **Expert Judgment**. The team had no historical velocity data and no prior projects of comparable scope, which rules out parametric methods such as COCOMO. Function Point Analysis was considered, but the requirement set changed materially during implementation, which would have made an early function count misleading. Expert Judgment was applied by estimating each level-3 deliverable against the closest task any team member had previously attempted, then adjusting upward for unfamiliarity with the specific technology. Estimates are in person-hours. The variance column is completed once actual effort is recorded.

W BS	Deliverable	Est. (h)	Actual (h)	Variance	Reasoning
1.1	Planning and Setup	24	26	+2	Proposal, legal review, stack decisions; mostly discussion rather than build
1.2	Backend Foundations	20	18	-2	Standard CRUD on a familiar stack; schema design straightforward
1.3	AI Conversation Engine	30	38	+8	Unfamiliar API; prompt authoring is iterative and hard to bound
1.4	Crisis Safety Layer	26	25	-1	Rule authoring is fast; test-set construction and false-positive tuning is not
1.5	Doctor System	40	44	+4	Authentication, queue logic, two new interfaces; largest single deliverable
1.6	Frontend	45	48	+3	Five screens plus landing page, bilingual, responsive, dark mode
1.7	Security Hardening	28	29	+1	Encryption and audit are documented patterns; integration testing is the cost
1.8	Testing and Closure	30	32	+2	Deployment, documentation, rehearsal
	Total	243	260	+17	

1.3 Risk Assessment and Mitigation Strategies

Likelihood and impact are rated Low / Medium / High. Risks marked "Occurred" materialised during the project; their mitigation column records what was actually done rather than what was planned. This approach ensures the risk register reflects operational reality rather than theoretical planning assumptions, making it a more useful artifact for future projects and for assessors evaluating the team's risk management maturity.

#	Risk	Likelihood	Impact	Mitigation	Status
R 1	AI provider quota exhausted during demonstration	High	High	Engine tries a high-volume model first, falls through to a second model, then to reviewed canned replies. The application degrades rather than erroring.	Occurred
R 2	Network blocks MongoDB port 27017	Medium	High	Diagnosed with a TCP connection test. Local MongoDB used for development; Atlas reserved for the deployed backend, which is unaffected by campus restrictions.	Occurred
R 3	Institutional account blocks generative AI APIs	Medium	Medium	API key created on a personal account once the restriction was identified.	Occurred
R 4	AI generates unsafe or inappropriate advice	Medium	High	Safety layer screens every message before the model is called and screens model output again before it reaches the user. System prompt forbids diagnosis and prescription.	Mitigated
R 5	Crisis phrase missed; user receives no helpline	Medium	High	Rule-based patterns in both languages, screened on every message regardless of interface language. Test set of 33 cases including code-mixed input.	Mitigated
R 6	False positives erode trust in the crisis banner	Medium	Medium	Word-boundary matching rather than substring. Test set includes deliberate traps such as "diet" and "cutting down on smoking".	Mitigated
R 7	Patient data exposed by database compromise	Low	High	Message content encrypted at rest with AES-256-GCM. No identity fields exist in the schema, so even decrypted content is not attributable to a person.	Mitigated

R 8	Doctor credentials brute-forced	Medium	High	Rate limiting per IP plus per-account lockout after six failures, which rotating addresses cannot evade. Passwords hashed with bcrypt cost 12.	Mitigated
R 9	Uneven contribution across the team	High	Medium	Roles assigned in the proposal; remaining work distributed by role in the final weeks.	Ongoing
R 10	Deployment left until the final week	High	High	Deployment surfaces environment problems that do not exist locally. Scheduled ahead of final documentation.	Ongoing
R 11	Scope growth beyond the original proposal	High	Medium	Voice input, bilingual support, doctor accounts and the security layer were added beyond proposal scope. Accepted deliberately; documented in Section 2.1 and Section 3.1.	Occurred
R 12	Work held on a single machine with no backup	Medium	High	Repository published to GitHub with all team members added as collaborators.	Mitigated

1.4 Cost Estimation

The project was delivered at zero monetary cost. Every item is listed, including those that cost nothing, so that the basis of the figure is visible rather than assumed. All software components run on free tiers, all hardware consists of team members' existing personal devices, and all human effort is volunteered student time with no salary cost.

Planned Budget

Item	Category	Planned (LKR)	Basis
MongoDB Atlas (M0 cluster)	Software	15000	Free tier, 512 MB - sufficient for prototype data volumes
Google Gemini API	Software	20000	Free tier assumed adequate for development and demonstration
Render (backend hosting)	Software	10000	Free web service tier
Vercel (frontend hosting)	Software	10000	Free hobby tier
GitHub (private repository)	Software	5000	Free for private repositories
Node.js, React, Express, mongoose	Software	5000	Open source

Development machines	Hardware	30000	Team members' existing personal laptops
Mobile test devices	Hardware	5000	Team members' existing phones
Human resources	Human	500 (243 person-hours)	Student project; no salary cost. Effort valued at zero but recorded in hours
	Total	221500	

Actual Budget

Item	Planned (LKR)	Actual (LKR)	Variance / Note
MongoDB Atlas	15000	4500	Free tier retained. Local MongoDB used for development after a network port restriction (I1)
Google Gemini API	20000	6000	Free tier retained, though daily quota limits were reached during testing (I4)
Render	10000	3500	[confirm on deployment]
Vercel	10000	2500	[confirm on deployment]
GitHub	5000	1500	Private repository on free tier
Hardware	30000	8000	No purchases required
Human resources	150000	121500	[record actual person-hours]
Total	240000	147500	

The zero monetary cost is a genuine finding rather than an omission: the platform was built entirely on free tiers. The meaningful cost is the 243 estimated person-hours. A production deployment would incur real costs - a paid database tier, paid AI usage at scale, a managed key service, and domain registration and these are noted in Section 3.4.

1.4 Resource Allocation

Resource	Type	Purpose	Used in
Node.js 24 / Express 5	Runtime, framework	HTTP API server	1.2, 1.5, 1.7
React 19 + Vite	Frontend framework	Component-based UI with fast builds	1.6

React Router	Library	Client-side routing across screens	1.6
MongoDB + Mongoose	Database	Document storage; schema validation enforces the anonymity design	1.2, 1.7
MongoDB Atlas	Cloud service	Production database for the deployed backend	1.8.3
Google Gemini API	Cloud service	AI conversation generation	1.3
bcryptjs	Library	Password hashing for doctor accounts	1.5.1
jsonwebtoken	Library	Doctor authentication and session ownership tokens	1.5.2, 1.7.4
express-rate-limit	Library	Brute-force and quota protection	1.7.3
Node crypto (AES-256-GCM)	Built-in module	Encryption of message content at rest	1.7.1
Web Speech API	Browser API	Patient voice input, transcribed on device	1.6.5
MediaRecorder API	Browser API	Doctor voice notes	1.5
lucide-react	Library	Interface icons as React components	1.6
Noto Sans Sinhala	Font	Sinhala script rendering	1.6.4
GitHub	Service	Version control and team collaboration	All
Render / Vercel	Cloud service	Backend and frontend hosting	1.8.3
Cloudflare Tunnel	Tool	Temporary public URLs for team testing	1.8.2
VS Code	Tool	Development environment	All
mongosh / MongoDB Compass	Tool	Database inspection and claim verification	1.2, 1.7
Personal laptops and phones	Hardware	Development and cross-device testing	All

1.5 Communication and Reporting Plan

Channels

Channel	Purpose	Participants
WhatsApp group	Day-to-day coordination, quick questions, sharing environment credentials	All members

GitHub	Code, version history, progress visible through commits	All members
In-person meetings	Design decisions, progress review, problem solving	All members
Google Drive	Shared documents, issue log, meeting minutes	All members
Email	Formal communication with the module lecturer	Project lead

Reporting and Documentation

- Progress is visible through commit history on GitHub, which records what changed, when, and by whom.
- Issues and their resolutions are logged in a shared spreadsheet (Section 2.1).
- Meeting decisions and action items are recorded as dated minutes (Section 2.2).
- Environment credentials are shared over WhatsApp and never committed to the repository, which is enforced by .gitignore.

2. Project Execution and Monitoring

2.1 Issue Tracking and Change Management

The table below records issues encountered during implementation. Several were environmental rather than defects in the code, which is itself a finding worth recording - a meaningful share of project time went on problems outside the software. Each issue is documented with its identification context, the team member assigned, the resolution applied, and the date of resolution, providing a complete audit trail of the project's problem-solving history.

#	Issue	Identified	Assigned	Resolution	Resolved
I 1	MongoDB Atlas unreachable; connection refused on port 27017	05 August 2026	Himasha Dushan	Diagnosed with Test-NetConnection, confirming the network blocked the port rather than a credential fault. MongoDB installed locally for development; Atlas retained for deployment.	07 August 2026

I 2	Gemini API returned 403 PERMISSION_DENIED	08 August 2026	Kavindu Mihiranga	Traced to the institutional Google Workspace account restricting generative AI APIs. Key regenerated on a personal account.	09 August 2026
I 3	AI replies truncated mid-sentence	11 August 2026	Kavindu Mihiranga	Output token limit too low; Gemini 3.x consumes part of the budget on internal reasoning. Limit raised and reply length controlled by the system prompt instead.	12 August 2026
I 4	Daily AI quota exhausted during testing	13 August 2026	Kavindu Mihiranga	Model fallback chain added: a high-volume model first, then a second model, then reviewed canned replies.	14 August 2026
I 5	AI continued replying after a doctor joined the conversation	18 August 2026	Himasha Dushan	Design flaw: the interface claimed a doctor was present while the model answered. A separate doctor endpoint was added and the client routes messages there once a doctor joins.	20 August 2026
I 6	Doctor unaware when a patient ended the conversation	21 August 2026	Himasha Dushan	The patient-end route cleared claimedBy, so the doctor's next poll failed the ownership check and returned 403, which the client swallowed silently. The session now stays attached until the doctor clears it.	22 August 2026
I 7	Sinhala text rendered incorrectly; headings distorted	19 August 2026	Chanuka Lakshan	Two causes: DM Serif Display contains no Sinhala glyphs, and the browser synthesised a faux italic for a font with no italic face. Font switched to Noto Sans Sinhala and italics disabled in Sinhala mode.	23 August 2026
I 8	Live session count showed zero during an active conversation	24 August 2026	Himasha Dushan	lastSeenAt was only updated by the doctor-status poll, which does not run during an AI chat. Sending a message now updates it, and the live window widened from 5 to 15 minutes.	25 August 2026
I 9	Chat write endpoint accepted a bare session ID	25 August 2026	H. C. Shakya	Reading a conversation was gated but writing was not, so anyone holding a session ID could post as the patient. Session ownership tokens applied to all five affected endpoints.	27 August 2026
I 10	Mongoose getters bypassed by .lean() queries	27 August 2026	Himasha Dushan	Three queries would have returned raw ciphertext to the client. An explicit decryption helper was added and applied to each.	27 August 2026

Changes to the Original Plan

Change	Reason	Impact
Doctor verification implemented rather than simulated	The proposal (section 9) described mock pre-approved profiles. The team chose to build real registration, administrator approval and a doctor dashboard.	Consumed the schedule slack earned in earlier weeks; displaced mood tracking
Voice input added	Not in the original scope. Added to lower the barrier for users who find typing about sensitive topics harder than speaking.	Patient speech is transcribed on-device and no audio is stored, preserving the anonymity design
Bilingual English/Sinhala support added	Not in the original scope. The platform targets Sri Lankan users, many of whom are not comfortable in English.	Required a bilingual crisis-detection layer, expanding the safety test set from 20 to 33 cases
Security layer added	Encryption at rest was promised in proposal section 9 but unimplemented. Audit logging, rate limiting and account lockout were added alongside it.	Approximately 28 person-hours not in the original estimate
Mood tracking deferred	Optional scope in proposal section 4, displaced by the changes above.	Documented as a deliberate deferral rather than an omission

2.2 Status Updates and Team Meetings

Meeting minutes should be dated and concise, recording decisions and action items rather than discussion. This section provides the template and guidance for completing the meeting record.

[TO COMPLETE] This section must be completed by the team from actual records. If formal minutes were not kept, reconstruct them from WhatsApp history and commit dates - and state in the document that this is what was done. Fabricated minutes are the most detectable form of padding in a Project Management submission; an honest reconstruction is both acceptable and more defensible.

Meeting minutes (Google Docs): _____

3. Project Closure Report

3.1 Summary of Project Execution

JustAsk LK was proposed as an anonymous, AI-first health support platform for people in Sri Lanka who avoid discussing sensitive health concerns because of stigma. The project ran over eight weeks from planning to submission, delivering a working platform with AI conversation, crisis detection, doctor escalation, bilingual support, and encryption at rest.

Planning Phase

The team defined the problem, established the ethical position, and reviewed the Sri Lankan legal context

- the Medical Ordinance and SLMC rules, the Personal Data Protection Act No. 9 of 2022, the Mental Diseases Ordinance, and the Computer Crimes Act. The central design decision emerged here: the platform would collect no identifying information at all, so that anonymity would be a property of the data model rather than a policy that could later be changed.

Implementation Phase

Development followed the architecture set out in the proposal: a React frontend, an Express API, a rule-based safety layer positioned between the request and the AI, and MongoDB storage keyed only on a random session identifier. Work progressed ahead of the original Gantt schedule; crisis-detection safety, planned for Week 5, was completed in Week 4, as were the AI engine and doctor escalation.

That slack was spent on scope beyond the proposal. Doctor verification, described in the proposal as simulated, was implemented as a working system with registration, administrator approval and a separate dashboard. Voice input and bilingual support were added. A security layer - encryption at rest, audit logging, rate limiting, per-account lockout and session ownership tokens - was built to close the gap between what proposal section 9 promised and what existed.

Milestones

Milestone	Planned	Actual	Note
Project proposal submitted	Week 1	03 August 2026	
Backend and database operational	Week 2	10 August 2026	Delayed by a network port restriction (I1)
AI chat functional	Week 3-4	17 August 2026	
Crisis detection passing test set	Week 5	20 August 2026	Ahead of schedule
Doctor system operational	Not in original plan	24 August 2026	Scope extension
Security layer complete	Not in original plan	26 August 2026	Scope extension

Repository published	Week 1	21 August 2026	Later than intended; single-machine risk carried until then
Deployed to public URL	Week 8	29 August 2026	
Final documentation	Week 8	31 August 2026	

Measured Outcome

The proposal set one quantified objective: correctly flag at least 90% of crisis-indicator phrases in an internal test set. The implemented safety layer passes 33 of 33 cases, including ten deliberate false-positive traps and two code-mixed English/Sinhala messages. The result is reproducible by running `node services/safetyLayer.test.js`.

3.2 Lessons Learned and Challenges Faced

Technical Challenges

- **Environment before code** - a meaningful share of project time went on problems that had nothing to do with the code: a blocked database port, an institutional account restricting the AI API, and DNS resolution differing between the operating system and Node. The lesson is that environment problems should be budgeted for explicitly rather than treated as unexpected.
- **The most serious defect was a design flaw, not a bug** - the AI continued answering after a doctor joined a conversation, so the interface said one thing while the system did another. It was found by using the product rather than by testing a function. Manual end-to-end use caught what unit-level thinking did not.
- **Silent failures are the dangerous ones** - Mongoose getters decrypt message content transparently, but `.lean()` queries bypass getters entirely, and three endpoints would have returned ciphertext to users. A convenience that works silently until one specific query pattern is used is a category of bug worth watching for.
- **Language support is more than strings** - adding Sinhala was not translation alone. The chosen fonts contained no Sinhala glyphs, the browser synthesised a faux italic that distorted the script, and letter-spacing tuned for English scattered Sinhala words. Localisation is a design problem, not a string-replacement problem.
- **Security decisions involve real trade-offs** - end-to-end encryption was considered and rejected, because a server that cannot read messages cannot run crisis detection on them. Choosing the weaker cryptographic property in order to keep the stronger safety property was the correct call, and being able to articulate why is more valuable than the implementation itself.

Teamwork Challenges

Our team divided the work according to individual technical roles, including backend development, AI and safety, frontend development, and UX/documentation. This worked well because each member could focus on their main responsibilities, while GitHub, WhatsApp, and regular discussions helped us coordinate the different components. However, the workload was not always perfectly balanced, and some integration, testing, and deployment tasks required additional effort from certain members during the later stages.

We also faced challenges such as database connectivity restrictions, AI API limitations, and integration issues between the frontend, backend, AI, and safety systems. Some of these problems were identified relatively late, which increased the pressure near the deadline. If we were doing the project again, we would integrate and test each component earlier, maintain clearer task ownership, and deploy a basic working version sooner. This would help us identify technical and teamwork issues earlier and distribute the workload more evenly.

Key Takeaways

- **Deploying early is not optional.** Deployment surfaces environment and configuration problems that do not exist locally, and leaving it until the final week compresses exactly the phase with the least slack.
- **Version control is a risk control, not a convenience.** The project existed on a single machine for longer than it should have; publishing to a shared repository should be the first task, not a later one.
- **Safety features need adversarial tests, not confirming ones.** The crisis test set is only credible because it includes cases that must not trigger.
- **Scope growth has a cost even when every addition is individually justified.** Each feature added also has to be tested, documented and demonstrated.

3.3 Team Reflections and Individual Contributions

The guideline is explicit that each member must write their own contribution and reflection, and that these must be specific rather than a shared generic description. The structure below is provided; the content must come from each member individually.

Kavindu Mihiranga - AI / Conversation Engineer

Tasks owned: WBS 1.3.1–1.3.5

I worked on the AI Conversation Engine covering WBS 1.3.1–1.3.5. I evaluated and selected the Gemini API as the LLM provider (1.3.1) and implemented the conversation logic in `services/aiEngine.js`. My work included creating the safety-focused system prompt, handling conversation history and context, tuning the tone according to the conversation stage, and implementing a fallback chain so that the system could continue operating when the primary AI model was unavailable.

One of the main difficulties was dealing with the AI API during development and testing. The institutional Google Workspace account initially restricted access to generative AI APIs, and later the daily AI quota was exhausted during testing. I also encountered responses being truncated because the output token limit was too low. I handled these issues by using an appropriate API key, increasing the output limit, controlling response length through the system prompt, and implementing a fallback chain with a second model and reviewed canned responses. In future, I would test API quotas and fallback behaviour much earlier and design the AI service around provider limitations from the beginning.

H. C. Shakya - Safety Engineer

Tasks owned: WBS 1.4.1–1.4.4, 1.7.3, 1.8.1

I developed the Crisis Safety Layer for WBS 1.4.1–1.4.4 and contributed to the security hardening work in WBS 1.7.3 and testing in WBS 1.8.1. I implemented English and Sinhala crisis patterns in `services/safetyLayer.js`, including helpline routing and special handling for users under 16. I also created and executed the test cases in `safetyLayer.test.js`. The final test set contained 33 cases, including deliberate false-positive cases and code-mixed English/Sinhala messages, and all 33 cases passed.

The most difficult part was balancing sensitive crisis detection with false-positive prevention. Simple keyword matching could incorrectly trigger on normal phrases, so I used word-boundary matching and included deliberate traps such as words that contain crisis-related terms. I also worked on rate limiting and account lockout using `rateLimit.js` and `loginGuard.js`. During testing, I identified that a chat write endpoint accepted only a session ID without properly checking ownership, so session ownership tokens were applied to the affected endpoints. If I were doing the project again, I would involve clinical and native-Sinhala review earlier and create a larger adversarial test set before integration.

Dinuvi Laksarani - Frontend / UI Developer

Tasks owned: WBS 1.5.4, 1.5.5, 1.6.1–1.6.3, 1.6.5, 1.6.6, 1.8.

I developed the main user and doctor-facing interfaces under WBS 1.5.4, 1.5.5, 1.6.1–1.6.3, 1.6.5, 1.6.6 and 1.8.2. This included the doctor dashboard in `DoctorDashboard.jsx`, administrator verification in `AdminDashboard.jsx`, the bilingual landing page in `public/landing.html`, age and topic selection screens, and the main chat interface in `ChatScreen.jsx`. I also implemented browser speech-to-text through `useSpeechInput.js`, responsive layouts and dark mode through `index.css`, and carried out cross-device and browser testing. The main challenge was making the interface work consistently across different screen sizes and browsers while supporting features such as voice input and the crisis banner. Some features behaved differently depending on the browser, so I tested the application on different devices

and adjusted the layout and UI behaviour. I also had to coordinate with the backend team to ensure that frontend actions matched the available API endpoints, especially around doctor sessions. If I were doing the project again, I would start cross-browser testing earlier and establish a clearer shared UI/component structure before implementing all the screens.

Himasha Dushan - Backend / Data Engineer

Tasks owned: WBS 1.1.3–1.1.5, 1.2.1–1.2.4, 1.5.1–1.5.3, 1.7.1, 1.7.2, 1.7.4, 1.7.5, 1.8.3

I developed most of the backend and data infrastructure for the project, covering WBS 1.1.3–1.1.5, 1.2.1–1.2.4, 1.5.1–1.5.3, 1.7.1, 1.7.2, 1.7.4, 1.7.5 and 1.8.3. I selected the technology stack and designed the system architecture, then implemented the Express server and MongoDB connection in `index.js`. I created the anonymous session and chat message schemas, including the 30-day expiry requirement. I also developed doctor registration and authentication, session queue and atomic claiming, AES-256-GCM encryption, audit logging, session ownership tokens, session timeout handling and the production deployment.

The main difficulties were related to integration and environment issues rather than only coding. MongoDB Atlas was initially unreachable because the network blocked port 27017, so I diagnosed the issue and used local MongoDB for development while keeping Atlas for deployment. I also discovered issues where `.lean()` queries bypassed Mongoose decryption getters and where doctor/patient session ownership did not behave correctly after a conversation ended. These were handled by adding explicit decryption helpers and improving the session ownership and endpoint logic. In future, I would deploy a basic backend much earlier and test the complete API flow continuously rather than leaving deployment and some integration testing until the later stages.

Chanuka Lakshan - UX and Documentation Lead

Tasks owned: WBS 1.1.2, 1.6.4, 1.8.4

I worked on the UX, localisation and documentation aspects of the project, covering WBS 1.1.2, 1.6.4 and 1.8.4. My responsibilities included the legal and ethical review covering areas such as PDPA, SLMC and the Medical Ordinance, Sinhala localisation and font handling through `LangContext.jsx`, and the final project documentation. I also helped ensure that the user experience was appropriate for a bilingual platform and that the project documentation accurately reflected the implementation, changes and lessons learned.

One of the main difficulties I faced was Sinhala rendering and localisation. The original font did not contain suitable Sinhala glyphs, and the browser's handling of italic text caused the Sinhala headings to appear distorted. I addressed this by switching to Noto Sans Sinhala and disabling italics when Sinhala mode was active. This showed me that localisation involves more than translating text because typography and layout also affect usability. If I were doing the project again, I would test Sinhala typography and localisation at the beginning of frontend development rather than waiting until the interface was mostly complete. I would also maintain the documentation continuously throughout development instead of completing some sections retrospectively.

Overall Teamwork Reflection

As a team, we divided the project according to our main technical roles and worked together through GitHub, WhatsApp, in-person discussions and shared documentation. Communication between the backend, AI, safety and frontend work was particularly important because changes in one area often affected another. For example, the AI conversation flow had to work together with the crisis safety layer, while the frontend and backend needed to coordinate the doctor session and chat functionality.

Although the team completed the main objectives, the workload was not always evenly distributed and some integration and deployment work happened later than ideal. We also experienced environmental problems such as database network restrictions and AI API limitations. These experiences taught us that earlier deployment, clearer task allocation and more frequent integration testing would have improved the project. Overall, the team adapted to problems as they appeared and successfully combined the different contributions into a working platform.

3.4 Future Recommendations

Process Improvements

- **Deploy and version-control from day one** - publishing the repository and deploying a skeleton application in Week 1 would have removed a single-machine risk and surfaced environment problems while there was time to absorb them.
- **Write the work breakdown before starting work** - the work breakdown in Section 1.1 was produced during documentation rather than during planning. Building it first would have made the distribution of work explicit from the start and made uneven contribution visible earlier.
- **Track effort as it is spent** - several estimates in Section 1.2 were produced retrospectively. Recording effort as work happens costs minutes and makes variance analysis meaningful.

Technical Improvements

- **Managed key service** - the encryption key is held in an environment variable, so anyone with server access can decrypt stored conversations. A managed key service such as AWS KMS, Google Cloud KMS or HashiCorp Vault would move the boundary from "read a file" to "compromise the running application", and would log every use of the key.
- **Real SLMC registry integration** - the SLMC registration number is checked manually by an administrator. Integration with the official register would remove both the manual step and the possibility of approving an unverified number.
- **WebSockets instead of polling** - doctor and patient clients poll every four seconds. WebSockets would reduce latency and server load, at the cost of a more complex deployment.
- **Individual administrator accounts** - a single shared key authenticates administrators. Named keys were added so the audit log can attribute actions, but proper accounts with individual credentials would be needed in production.

- **Clinical review of the Sinhala patterns** - the crisis patterns were written by the team. A native-speaker review, and ideally clinical review of both the patterns and the crisis response wording, would be required before real deployment.

Feature Extensions

- **Mood tracking** - the one optional-scope item from the original proposal that was not built.
- **Doctor notifications** - doctors currently must keep the dashboard open to notice a waiting patient. An SMS or push notification would make the queue practical outside a demonstration.
- **Tamil language support** - Tamil support would extend the platform to a substantial population the current bilingual implementation does not serve.
- **Anomaly detection on audit data** - the audit log records that a doctor opened a conversation, but nothing acts on unusual patterns. Alerting on anomalous access would make the log preventive rather than only evidential.

4. Appendices

Appendix A – Links

- GitHub repository: <https://github.com/viperZ20/justask-lk.git>
- Deployed application: <http://justask-8w45ayqcm-viperz20.vercel.app/>
- Deployed API: <https://justask-lk-api.onrender.com/>
- Issue log (Google Sheets):
- Meeting minutes:

Date: 03 August 2026

- *Discussed the project idea and requirements.*
- *Divided tasks among team members.*
- *Planned the development work.*

Date: 17 August 2026

- *Reviewed the project progress.*
- *Discussed technical issues and possible solutions.*
- *Planned the remaining development work.*

Date: 27 August 2026

- *Reviewed the completed features.*
- *Discussed final testing and documentation.*
- *Planned the final submission.*

Appendix B - Supporting Documents

- Project proposal (submitted separately)
- Crisis detection test output - reproducible via `node services/safetyLayer.test.js`

Appendix C - Verifying the Anonymity and Encryption Claims

Both claims made in this report can be checked directly against the running system, which is why they are stated as findings rather than assurances:

- **Anonymity** - query the sessions collection. Only `sessionId`, `ageBand`, `topic`, `lang`, `status` and `timestamps` exist - there is no field capable of holding a name, NIC, email or phone number.
- **Encryption** - query the chatmessages collection. Stored content begins with the prefix `enc:v1:` followed by base64 ciphertext, and is unreadable without the server key.